MON APPRENTISSAGE — SERIES PYTHON

Python

Du zéro à l'expert

Chapitre 7 — Les Chaînes de Caractères

Maîtriser le texte — l'art de manipuler les strings

Auteur :

KETOTSA AMÉVI CLAUDE

Date de publication :

5 mars 2026

Publié dans le cadre de mon parcours d'apprentissage Python

Table des matières

Chapitre 7 — Les Chaînes de Caractères **2**
0.1 Créer une chaîne de caractères 2
0.2 Accéder aux caractères — L'indexation 2
0.3 Le slicing — Découper une chaîne 3
0.4 Les chaînes sont immuables 4
0.5 Les méthodes de transformation 5
0.6 Diviser et assembler 6
0.7 Vérifier le contenu d'une chaîne 7
0.8 Aligner et remplir des chaînes 7
0.9 Les chaînes multilignes et la documentation 8
0.10 Ce qui m'a le plus surpris dans ce chapitre 8
0.11 Résumé du Chapitre 7 9

Chapitre 7 — Les Chaînes de Caractères

À retenir

Les chaînes de caractères (`str`) sont omniprésentes en Python. Noms, messages, fichiers, URLs, données JSON — tout passe par le texte. Ce chapitre couvre en profondeur comment **créer, accéder, découper et transformer** les chaînes. Maîtriser les strings, c'est maîtriser une partie essentielle de Python.

Créer une chaîne de caractères

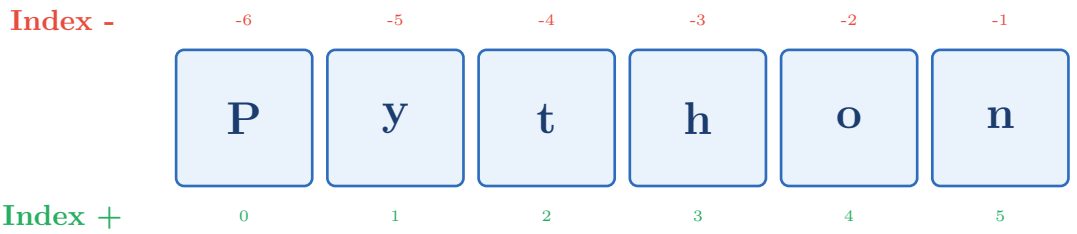
Une chaîne peut être délimitée par des guillemets simples, doubles, ou triples.

Les différentes façons de créer une chaîne

```python
# Guillemets simples
prenom = 'Alice'

# Guillemets doubles
message = "Bonjour tout le monde !"

# Guillemets doubles si apostrophe dans le texte
phrase = "J'apprends Python aujourd'hui."

# Guillemets simples si guillemets dans le texte
citation = 'Il a dit : "Bravo !"'

# Triple guillemets     chaine multiligne
texte_long = """Ceci est une chaine
qui s'etend sur
plusieurs lignes."""
print(texte_long)

# Triple guillemets simples     identique
poeme = '''Ligne un,
Ligne deux,
Ligne trois.'''
```

Accéder aux caractères — L'indexation

Une chaîne est une **séquence ordonnée** de caractères. Chaque caractère a un indice (index) qui commence à **0**.



Accéder aux caractères par index

```python
mot = "Python"

# Index positifs (de gauche a droite)
print(mot[0])     # P
print(mot[1])     # y
print(mot[5])     # n

# Index negatifs (de droite a gauche)
print(mot[-1])    # n  (dernier caractere)
print(mot[-2])    # o
print(mot[-6])    # P  (premier caractere)

# Longueur de la chaine
print(len(mot))   # 6
```

Erreur fréquente

Accéder à un index qui n'existe pas lève une `IndexError`. Pour une chaîne de longueur 6, les index valides sont de `0` à `5` (ou `-1` à `-6`). `mot[6]` provoquera une erreur immédiate.

Le slicing — Découper une chaîne

Le **slicing** permet d'extraire une portion d'une chaîne en spécifiant un début, une fin et un pas.

Slicing — syntaxe : chaine[debut :fin :pas]

```python
mot = "Python"

# chaine[debut:fin]      de debut a fin-1
print(mot[0:3])     # Pyt
print(mot[2:5])     # tho
print(mot[1:4])     # yth

# Omettre le debut ou la fin
print(mot[:3])      # Pyt  (du debut jusqu'a 3)
print(mot[3:])      # hon  (de 3 jusqu'a la fin)
print(mot[:])       # Python (copie complete)
```

```python
# Avec un pas
print(mot[::2])     # Pto  (un caractere sur deux)
print(mot[1::2])    # yhn

# Inverser une chaine      astuce classique Python
print(mot[::-1])    # nohtyP

# Exemples pratiques
email = "alice@example.com"
print(email[:5])            # alice
print(email[-3:])           # com
print(email[6:13])          # example
```

Ce qui m'a résisté

La syntaxe `[::-1]` pour inverser une chaîne m'a d'abord semblé magique — et un peu opaque. En décomposant : `:` signifie "du début à la fin", et `-1` est le pas, ce qui signifie "aller à reculons". Une fois que j'ai compris la logique `[debut:fin:pas]`, tout est devenu clair. C'est l'une des syntaxes les plus élégantes de Python — concise et expressive à la fois.

Les chaînes sont immuables

En Python, une chaîne ne peut **pas être modifiée** après sa création. On ne peut pas changer un caractère à la volée.

Immuabilité des chaînes

```python
mot = "Python"

#     Impossible      leve une TypeError
# mot[0] = "J"

#     On cree une NOUVELLE chaine
nouveau_mot = "J" + mot[1:]
print(nouveau_mot)    # Jython

# Toutes les methodes str retournent une NOUVELLE chaine
texte = "bonjour"
texte_maj = texte.upper()
print(texte)          # bonjour  (inchange !)
print(texte_maj)      # BONJOUR  (nouvelle chaine)
```

À retenir

L'immuabilité des chaînes est fondamentale. Quand on applique une méthode comme `.upper()` ou `.replace()`, on ne modifie **jamais** la chaîne originale — on obtient

une **nouvelle chaîne**. Il faut toujours assigner le résultat à une variable pour ne pas le perdre.

Les méthodes de transformation

Python offre une riche bibliothèque de méthodes pour transformer les chaînes.

Changer la casse

```python
texte = "Bonjour Tout Le Monde"

print(texte.upper())        # BONJOUR TOUT LE MONDE
print(texte.lower())        # bonjour tout le monde
print(texte.capitalize())   # Bonjour tout le monde
print(texte.title())        # Bonjour Tout Le Monde
print(texte.swapcase())     # bONJOUR tOUT lE mONDE
```

Nettoyer les espaces

```python
texte = "   Bonjour Python !   "

print(texte.strip())     # "Bonjour Python !"  (des deux cotes)
print(texte.lstrip())    # "Bonjour Python !   " (gauche)
print(texte.rstrip())    # "   Bonjour Python !" (droite)

# Tres utile pour nettoyer les saisies utilisateur
saisie = input("Votre prenom : ")
saisie_propre = saisie.strip().capitalize()
print(f"Bonjour, {saisie_propre} !")
```

Remplacer et chercher

```python
phrase = "J'aime le cafe et le the."

# replace()      remplacer des occurrences
print(phrase.replace("le", "la"))
#      J'aime la cafe et la the.

print(phrase.replace("cafe", "Python", 1))
#      J'aime le Python et le the.

# find()      trouver la position (retourne -1 si absent)
print(phrase.find("cafe"))     # 10
print(phrase.find("python"))   # -1

# count()      compter les occurrences
print(phrase.count("le"))      # 2
```

```python

# in      verifier la presence
print("cafe" in phrase)        # True
print("Python" in phrase)      # False
```

Diviser et assembler

split() et join() — couper et assembler

```python
# split()      diviser une chaine en liste
phrase = "Python est un langage puissant"
mots = phrase.split()
print(mots)
#     ['Python', 'est', 'un', 'langage', 'puissant']

# split() avec un separateur personnalise
date = "2024-03-15"
parties = date.split("-")
print(parties)           # ['2024', '03', '15']
print(parties[0])        # 2024
print(parties[2])        # 15

# join()      assembler une liste en chaine
mots = ["Python", "est", "incroyable"]
print(" ".join(mots))         # Python est incroyable
print("-".join(mots))         # Python-est-incroyable
print("".join(mots))          # Pythonestincroyable

# Cas pratique : inverser les mots d'une phrase
phrase = "Je apprends Python"
inverse = " ".join(phrase.split()[::-1])
print(inverse)    # Python apprends Je
```

Ce qui m'a résisté

`join()` m'a d'abord semblé à l'envers. On écrit `séparateur.join(liste)` et non `liste.join(séparateur)` — ce qui semblerait plus logique. La raison : `join()` est une méthode de la chaîne séparateur, pas de la liste. Une fois cette inversion mentale faite, c'est une des méthodes les plus puissantes pour assembler du texte proprement.

Vérifier le contenu d'une chaîne

Méthodes de vérification

```python
# isdigit()      ne contient que des chiffres
print("12345".isdigit())     # True
print("123a5".isdigit())     # False

# isalpha()      ne contient que des lettres
print("Python".isalpha())    # True
print("Python3".isalpha())   # False

# isalnum()      lettres et chiffres uniquement
print("Python3".isalnum())   # True
print("Python 3".isalnum()) # False (espace interdit)

# startswith() / endswith()
url = "https://www.python.org"
print(url.startswith("https"))    # True
print(url.endswith(".org"))       # True
print(url.endswith(".com"))       # False

# isupper() / islower()
print("PYTHON".isupper())    # True
print("python".islower())    # True
print("Python".isupper())    # False
```

Aligner et remplir des chaînes

center(), ljust(), rjust(), zfill()

```python
texte = "Python"

# Centrer dans une largeur donnee
print(texte.center(20))           #         Python
print(texte.center(20, "="))      # =======Python=======

# Aligner a gauche / droite
print(texte.ljust(20, "."))       # Python..............
print(texte.rjust(20, "."))       # ..............Python

# zfill()      remplir avec des zeros (utile pour les nombres)
numero = "42"
print(numero.zfill(6))            # 000042

# Application : afficher un tableau aligne
produits = [("Cafe", 2.50), ("Croissant", 1.20), ("Jus", 3.00)]
print(f"{'Produit':<15} {'Prix':>8}")
print("-" * 24)
```

```python
for nom, prix in produits:
    print(f"{nom:<15} {prix:>8.2f}    ")
```

Les chaînes multilignes et la documentation

Chaînes multilignes et docstrings

```python
# Chaine multiligne avec triple guillemets
message = """
Cher utilisateur,

Bienvenue sur notre plateforme.
Votre compte a ete cree avec succes.

Cordialement,
L'equipe Python
"""
print(message)

# Les docstrings      documenter son code
def calculer_aire(rayon):
    """
    Calcule l'aire d'un cercle.

    Parametre :
        rayon (float) : le rayon du cercle

    Retourne :
        float : l'aire du cercle
    """
    return 3.14159 * rayon ** 2

# Acceder a la documentation
print(calculer_aire.__doc__)
```

Ce qui m'a le plus surpris dans ce chapitre

Ce qui m'a résisté

Trois révélations de ce chapitre :

1. Les chaînes sont immuables — et c'est une force. Au début, ne pas pouvoir modifier `texte[0] = "X"` m'a frustré. Puis j'ai compris : l'immuabilité garantit que personne ne modifie accidentellement une chaîne partagée dans un programme. C'est une protection, pas une limitation. Chaque transformation produit une chaîne propre et nouvelle.

2. `join()` s'écrit à l'envers de ce qu'on attend. `" ".join(liste)` au lieu de `liste.join(" ")` — cette inversion m'a coûté plusieurs erreurs. La logique est que le séparateur *possède* la méthode. Bizarre au premier abord, mais cohérent avec la philosophie orientée objet de Python où tout est objet — y compris les chaînes.

3. Le slicing `[::-1]` pour inverser. Une seule expression de trois caractères pour inverser n'importe quelle séquence. Ni boucle, ni fonction externe. C'est la première fois que j'ai vraiment ressenti la **puissance expressive** de Python — faire beaucoup avec très peu.

Résumé du Chapitre 7

Ce que j'ai appris dans ce chapitre :

- Créer des chaînes avec guillemets simples, doubles ou triples.
- **Indexation** — accéder à un caractère avec `mot[0]` ou `mot[-1]`.
- **Slicing** — découper avec `[debut:fin:pas]`, inverser avec `[::-1]`.
- Les chaînes sont **immuables** — toute méthode retourne une nouvelle chaîne.
- Transformer : `upper()`, `lower()`, `strip()`, `replace()`.
- Diviser et assembler : `split()` et `join()`.
- Vérifier : `isdigit()`, `startswith()`, `endswith()`, `in`.
- Aligner : `center()`, `ljust()`, `rjust()`, `zfill()`.

"Le texte est partout. Maîtriser les strings, c'est maîtriser la communication entre l'humain et la machine."